FULL STACK APPLICATION DEVELOPMENT (week 2)

SESSION-6

1. Create a simple trigger BEFORE INSERT or update or delete trigger using the CREATE TRIGGER statement for driver schema assuming the otp for entry.
   Create table:

   TABLE:
   ```
   CREATE TABLE driver (
      driver_id INT PRIMARY KEY,
      otp INT
   );
   ```

   ```
   mysql> CREATE TABLE driver (
       ->     driver_id INT PRIMARY KEY,
       ->     otp INT
       -> );
   Query OK, 0 rows affected (0.02 sec)
   ```

   OTP must be exactly 4 digits:

   ```
   DELIMITER //

   CREATE TRIGGER otp_validation
   BEFORE INSERT ON driver
   FOR EACH ROW
   BEGIN
      IF NEW.otp < 1000 OR NEW.otp > 9999 THEN
         SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'OTP must be exactly 4 digits';
      END IF;
   END //

   DELIMITER ;
   ```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER otp_validation
    -> BEFORE INSERT ON driver
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.otp < 1000 OR NEW.otp > 9999 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'OTP must be exactly 4 digits';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

2. Create a simple trigger AFTER INSERT When a customer adds a dish to their order, the system must automatically calculate the price and add it to the order's running total.

   TABLE:

   CREATE TABLE orders (
   order_id INT PRIMARY KEY,
   total_amount INT DEFAULT 0
   );

   CREATE TABLE order_items (
   item_id INT PRIMARY KEY,
   order_id INT,
   price INT
   );

```
mysql> CREATE TABLE orders (
    ->     order_id INT PRIMARY KEY,
    ->     total_amount INT DEFAULT 0
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> CREATE TABLE order_items (
    ->     item_id INT PRIMARY KEY,
    ->     order_id INT,
    ->     price INT
    -> );
Query OK, 0 rows affected (0.01 sec)
```

Trigger:

```
DELIMITER //

CREATE TRIGGER update_order_total
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
  UPDATE orders
  SET total_amount = total_amount + NEW.price
  WHERE order_id = NEW.order_id;
END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER update_order_total
    -> AFTER INSERT ON order_items
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE orders
    ->     SET total_amount = total_amount + NEW.price
    ->     WHERE order_id = NEW.order_id;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

3. Automatically set created_at timestamp.

TABLE:

```
CREATE TABLE users (
user_id INT PRIMARY KEY,
name VARCHAR(30),
```

created_at DATETIME

);

```
mysql> CREATE TABLE users (
    ->     user_id INT PRIMARY KEY,
    ->     name VARCHAR(30),
    ->     created_at DATETIME
    -> );
Query OK, 0 rows affected (0.01 sec)
```

<u>Trigger:</u>

DELIMITER //

CREATE TRIGGER set_created_at

BEFORE INSERT ON users

FOR EACH ROW

BEGIN

SET NEW.created_at = NOW();

END //

DELIMITER ;

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER set_created_at
    -> BEFORE INSERT ON users
    -> FOR EACH ROW
    -> BEGIN
    ->     SET NEW.created_at = NOW();
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

4. Prevent negative salary entry by Stop inserting or updating an employee record if salary < 0.

TABLE:

```
CREATE TABLE employee (
  emp_id INT PRIMARY KEY,
  name VARCHAR(30),
  salary INT
);
```

```
mysql> CREATE TABLE employee (
    ->     emp_id INT PRIMARY KEY,
    ->     name VARCHAR(30),
    ->     salary INT
    -> );
Query OK, 0 rows affected (0.01 sec)
```

Trigger:

```
DELIMITER //

CREATE TRIGGER salary_check
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
  IF NEW.salary < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot be negative';
  END IF;
END //

DELIMITER ;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER salary_check
    -> BEFORE INSERT ON employee
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.salary < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Salary cannot be negative';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

Same rule for UPDATE:

```
DELIMITER //


CREATE TRIGGER salary_update_check

BEFORE UPDATE ON employee

FOR EACH ROW

BEGIN

  IF NEW.salary < 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Salary cannot be negative';

  END IF;

END //
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER salary_update_check
    -> BEFORE UPDATE ON employee
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.salary < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Salary cannot be negative';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

1. Maintain audit log table: Whenever an employee record is updated, store old values in an employee_audit table.

   Table:

   CREATE TABLE employee_audit (

   emp_id INT,

   old_salary INT,

   changed_at DATETIME

   );

```
mysql>
mysql> DELIMITER ;
mysql> CREATE TABLE employee_audit (
    ->     emp_id INT,
    ->     old_salary INT,
    ->     changed_at DATETIME
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Trigger:

DELIMITER //

CREATE TRIGGER employee_audit_log

BEFORE UPDATE ON employee

FOR EACH ROW

BEGIN

INSERT INTO employee_audit

VALUES (OLD.emp_id, OLD.salary, NOW());

END //

DELIMITER ;

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER employee_audit_log
    -> BEFORE UPDATE ON employee
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO employee_audit
    ->     VALUES (OLD.emp_id, OLD.salary, NOW());
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

SESSION-7

1. Find number of days between start_date and end_date.

   SELECT DATEDIFF(end_date, start_date) AS no_of_days

   FROM project;

```
mysql> SELECT DATEDIFF(end_date, start_date) AS no_of_days
    -> FROM project;
+------------+
| no_of_days |
+------------+
|          9 |
|         15 |
|         14 |
+------------+
3 rows in set (0.00 sec)
```

2. Find the expiry date after 30 days.
   SELECT DATE_ADD(start_date, INTERVAL 30 DAY) AS expiry_date
   FROM subscription;

```
mysql> CREATE TABLE subscription (
    ->     sub_id INT PRIMARY KEY,
    ->     start_date DATE
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO subscription VALUES
    -> (101, '2025-02-01'),
    -> (102, '2025-03-10'),
    -> (103, '2025-04-05');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT DATE_ADD(start_date, INTERVAL 30 DAY) AS expiry_date
    -> FROM subscription;
+-------------+
| expiry_date |
+-------------+
| 2025-03-03  |
| 2025-04-09  |
| 2025-05-05  |
+-------------+
3 rows in set (0.00 sec)
```

3. Get Date before seven days from today.
   SELECT DATE_SUB(CURDATE(), INTERVAL 7 DAY) AS date_before_7_days;

```
mysql> CREATE TABLE activity_log (
    ->     log_id INT PRIMARY KEY,
    ->     created_on DATE
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO activity_log VALUES
    -> (301, '2025-02-08'), -- Saturday
    -> (302, '2025-02-09'), -- Sunday
    -> (303, '2025-02-11'),
    -> (304, '2025-02-15'); -- Saturday
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT DATE_SUB(CURDATE(), INTERVAL 7 DAY) AS date_before_7_days;
+--------------------+
| date_before_7_days |
+--------------------+
| 2026-01-27         |
+--------------------+
1 row in set (0.00 sec)
```

4. Get the records created on weekend dates.
   SELECT *
   FROM activity_log
   WHERE DAYOFWEEK(created_on) IN (1, 7);

```
mysql> CREATE TABLE date_records (
    ->     record_id INT PRIMARY KEY,
    ->     input_date VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO date_records VALUES
    -> (1, '15-08-2025'),
    -> (2, '01-01-2024'),
    -> (3, '26-01-2025');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT *
    -> FROM activity_log
    -> WHERE DAYOFWEEK(created_on) IN (1, 7);
+--------+------------+
| log_id | created_on |
+--------+------------+
|    301 | 2025-02-08 |
|    302 | 2025-02-09 |
|    304 | 2025-02-15 |
+--------+------------+
3 rows in set (0.00 sec)
```

5. Read input date as string then convert to date format 'DD-MM-YYYY' then display the month name from the date given.

```
SELECT record_id,
    input_date,
    MONTHNAME(
      STR_TO_DATE(input_date, '%d-%m-%Y')
    ) AS month_name
FROM date_records;
```

```
mysql> SELECT record_id,
    ->        input_date,
    ->        MONTHNAME(
    ->            STR_TO_DATE(input_date, '%d-%m-%Y')
    ->        ) AS month_name
    -> FROM date_records;
+----------+------------+------------+
| record_id | input_date | month_name |
+----------+------------+------------+
|         1 | 15-08-2025 | August     |
|         2 | 01-01-2024 | January    |
|         3 | 26-01-2025 | January    |
+----------+------------+------------+
3 rows in set (0.00 sec)
```

SESSION-8

1. Improve search performance on email column using CREATE INDEX.

   CREATE INDEX idx_email
   ON users_account(email);
   SELECT * FROM users_account WHERE email = 'arun@gmail.com';

```
mysql> CREATE INDEX idx_email
    -> ON users_account(email);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM users_account WHERE email = 'arun@gmail.com';
+---------+----------+----------------+--------------+--------+---------------------+
| user_id | username | email          | aadhar       | status | created_at          |
+---------+----------+----------------+--------------+--------+---------------------+
|       1 | arun01   | arun@gmail.com | 123456789012 | active | 2026-02-03 10:25:40 |
+---------+----------+----------------+--------------+--------+---------------------+
1 row in set (0.00 sec)
```

2. Create UNIQUE INDEX to ensure no duplicate values in username.

   CREATE UNIQUE INDEX idx_username_unique
   ON users_account(username);

```
mysql> CREATE UNIQUE INDEX idx_username_unique
    -> ON users_account(username);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3. Find and remove the unused index from the table using DROP INDEX.

   <u>View indexes:</u>
   SHOW INDEX FROM users_account;

```
mysql> SHOW INDEX FROM users_account;
+---------------+------------+---------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
+---------------+---------+------------+
| Table         | Non_unique | Key_name            | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment
| Index_comment | Visible | Expression |
+---------------+------------+---------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
+---------------+---------+------------+
| users_account |          0 | PRIMARY             |            1 | user_id     | A         |           3 |     NULL |   NULL |      | BTREE      |
|               | YES     | NULL       |
| users_account |          0 | idx_username_unique |            1 | username    | A         |           3 |     NULL |   NULL | YES  | BTREE      |
|               | YES     | NULL       |
| users_account |          1 | idx_email           |            1 | email       | A         |           3 |     NULL |   NULL | YES  | BTREE      |
|               | YES     | NULL       |
+---------------+------------+---------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------
+---------------+---------+------------+
3 rows in set (0.03 sec)
```

   <u>Drop unused index:</u>
   DROP INDEX idx_email ON users_account;

```
mysql> DROP INDEX idx_email ON users_account;
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4. A SELECT query using WHERE aadhar= ? is slow. Fix it using index.

   CREATE INDEX idx_aadhar
   ON users_account(aadhar);

   SELECT * FROM users_account WHERE aadhar = '123456789012';

```
mysql> CREATE INDEX idx_aadhar
    -> ON users_account(aadhar);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM users_account WHERE aadhar = '123456789012';
+---------+----------+----------------+--------------+--------+---------------------+
| user_id | username | email          | aadhar       | status | created_at          |
+---------+----------+----------------+--------------+--------+---------------------+
|       1 | arun01   | arun@gmail.com | 123456789012 | active | 2026-02-03 10:25:40 |
+---------+----------+----------------+--------------+--------+---------------------+
1 row in set (0.00 sec)
```

5. Create a composite index: (status, created_at) to query filters by status and sorts by created_at.

   CREATE INDEX idx_status_created
   ON users_account(status, created_at);

   SELECT *
   FROM users_account
   WHERE status = 'active'
   ORDER BY created_at;

```
mysql> CREATE INDEX idx_status_created
    -> ON users_account(status, created_at);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
    -> FROM users_account
    -> WHERE status = 'active'
    -> ORDER BY created_at;
+---------+----------+------------------+--------------+--------+---------------------+
| user_id | username | email            | aadhar       | status | created_at          |
+---------+----------+------------------+--------------+--------+---------------------+
|       1 | arun01   | arun@gmail.com   | 123456789012 | active | 2026-02-03 10:25:40 |
|       3 | chitra03 | chitra@gmail.com | 345678901234 | active | 2026-02-03 10:25:40 |
+---------+----------+------------------+--------------+--------+---------------------+
2 rows in set (0.00 sec)
```

SESSION-9

1. Extract the letters before @ from the username and append 123 at the end to create a default password.

```
SELECT email,
    CONCAT(
      SUBSTRING_INDEX(email, '@', 1),
      '123'
    ) AS default_password
FROM user_account;
```

```
mysql> SELECT email,
    ->        CONCAT(
    ->            SUBSTRING_INDEX(email, '@', 1),
    ->            '123'
    ->        ) AS default_password
    -> FROM user_account;
+--------------------+------------------+
| email              | default_password |
+--------------------+------------------+
| arun@gmail.com     | arun123          |
| bala@yahoo.com     | bala123          |
| chitra@outlook.com | chitra123        |
+--------------------+------------------+
3 rows in set (0.00 sec)
```

2. Correct the invalid contact numbers which contains spaces, dashes or any other symbols. Keep only numbers to make the contact valid.

```
SELECT contact,
    REGEXP_REPLACE(contact, '[^0-9]', '') AS valid_contact
FROM user_account;
```

```
mysql> SELECT contact,
    ->        REGEXP_REPLACE(contact, '[^0-9]', '') AS valid_contact
    -> FROM user_account;
+--------------+---------------+
| contact      | valid_contact |
+--------------+---------------+
| 98765-43210  | 9876543210    |
| 044-234 5678 | 0442345678    |
| NULL         | NULL          |
+--------------+---------------+
3 rows in set (0.01 sec)
```

3. Normalise the employee names given in mixed case using UPPER(), LOWER(), SUBSTRING(), CONCAT() functions.

```
SELECT emp_name,
    CONCAT(
      UPPER(SUBSTRING(emp_name,1,1)),
      LOWER(SUBSTRING(emp_name,2))
    ) AS normalized_name
FROM employee_details;
```

```
mysql> SELECT emp_name,
    ->        CONCAT(
    ->            UPPER(SUBSTRING(emp_name,1,1)),
    ->            LOWER(SUBSTRING(emp_name,2))
    ->        ) AS normalized_name
    -> FROM employee_details;
+------------+-----------------+
| emp_name   | normalized_name |
+------------+-----------------+
| aRun kuMaR | Arun kumar      |
| MEENA devi | Meena devi      |
| raJesH     | Rajesh          |
+------------+-----------------+
3 rows in set (0.00 sec)
```

4. Handle multiple fallback columns in the use table with the priority of contact details as Mobile→Phone→email.

```
SELECT username,
    COALESCE(mobile, phone, email) AS preferred_contact
FROM user_account;
```

```
mysql> SELECT username,
    ->        COALESCE(mobile, phone, email) AS preferred_contact
    -> FROM user_account;
+----------+--------------------+
| username | preferred_contact  |
+----------+--------------------+
| arun     | 98765-43210        |
| bala     | 044-234 5678       |
| chitra   | chitra@outlook.com |
+----------+--------------------+
3 rows in set (0.00 sec)
```

5. Find average experience per department using the functions AVG(), TIMESTAMPDIFF(), GROUP BY.

```
SELECT department,
    AVG(
      TIMESTAMPDIFF(YEAR, join_date, CURDATE())
    ) AS avg_experience_years
FROM employee_details
GROUP BY department;
```

```
mysql> SELECT department,
    ->        AVG(
    ->            TIMESTAMPDIFF(YEAR, join_date, CURDATE())
    ->        ) AS avg_experience_years
    -> FROM employee_details
    -> GROUP BY department;
+------------+----------------------+
| department | avg_experience_years |
+------------+----------------------+
| IT         |               8.0000 |
| HR         |               6.0000 |
+------------+----------------------+
2 rows in set (0.00 sec)
```

SESSION-10

1. Create a function to calculate net salary after 10% tax deduction.

```
DELIMITER //
CREATE FUNCTION net_salary(salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  RETURN salary - (salary * 0.10);
END //
DELIMITER ;

SELECT emp_id, salary, net_salary(salary) AS net_salary
FROM emp_salary;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION net_salary(salary DECIMAL(10,2))
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->     RETURN salary - (salary * 0.10);
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT emp_id, salary, net_salary(salary) AS net_salary
    -> FROM emp_salary;
+--------+------------+------------+
| emp_id | salary     | net_salary |
+--------+------------+------------+
|      1 |  250000.00 |  225000.00 |
|      2 |  450000.00 |  405000.00 |
|      3 |  750000.00 |  675000.00 |
|      4 | 1200000.00 | 1080000.00 |
+--------+------------+------------+
4 rows in set (0.00 sec)
```

2. Create a function to check activity of the user. Return Active if last login within 30 days, else Inactive.

```
DELIMITER //

CREATE FUNCTION user_status(last_login DATE)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
  IF DATEDIFF(CURDATE(), last_login) <= 30 THEN
    RETURN 'Active';
  ELSE
    RETURN 'Inactive';
  END IF;
END //

DELIMITER ;

SELECT user_id, user_status(last_login) AS status
FROM user_activity;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION user_status(last_login DATE)
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> BEGIN
    ->     IF DATEDIFF(CURDATE(), last_login) <= 30 THEN
    ->         RETURN 'Active';
    ->     ELSE
    ->         RETURN 'Inactive';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT user_id, user_status(last_login) AS status
    -> FROM user_activity;
+---------+----------+
| user_id | status   |
+---------+----------+
|     101 | Active   |
|     102 | Inactive |
+---------+----------+
2 rows in set (0.00 sec)
```

3. Create a function to perform dynamic tax slab calculation based on the following categories:

```
DELIMITER //

CREATE FUNCTION calculate_tax(salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  IF salary <= 300000 THEN
    RETURN 0;
  ELSEIF salary <= 600000 THEN
    RETURN salary * 0.10;
  ELSEIF salary <= 1000000 THEN
    RETURN salary * 0.20;
  ELSE
    RETURN salary * 0.30;
  END IF;
END //
```

```
DELIMITER ;

SELECT emp_id, salary, calculate_tax(salary) AS tax_amount
FROM emp_salary;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION calculate_tax(salary DECIMAL(10,2))
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->     IF salary <= 300000 THEN
    ->         RETURN 0;
    ->     ELSEIF salary <= 600000 THEN
    ->         RETURN salary * 0.10;
    ->     ELSEIF salary <= 1000000 THEN
    ->         RETURN salary * 0.20;
    ->     ELSE
    ->         RETURN salary * 0.30;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT emp_id, salary, calculate_tax(salary) AS tax_amount
    -> FROM emp_salary;
+--------+------------+------------+
| emp_id | salary     | tax_amount |
+--------+------------+------------+
|      1 |  250000.00 |       0.00 |
|      2 |  450000.00 |   45000.00 |
|      3 |  750000.00 |  150000.00 |
|      4 | 1200000.00 |  360000.00 |
+--------+------------+------------+
4 rows in set (0.00 sec)
```

4. Create a function to categorize the employees based on the years of experience:

```
DELIMITER //

CREATE FUNCTION experience_category(join_date DATE)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
  DECLARE years INT;
  SET years = TIMESTAMPDIFF(YEAR, join_date, CURDATE());

  IF years < 2 THEN
    RETURN 'Fresher';
  ELSEIF years BETWEEN 2 AND 5 THEN
    RETURN 'Junior';
  ELSEIF years BETWEEN 6 AND 10 THEN
```

```
    RETURN 'Mid';
  ELSE
    RETURN 'Senior';
  END IF;
END //

DELIMITER ;

SELECT emp_id, experience_category(join_date) AS category
FROM emp_salary;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION experience_category(join_date DATE)
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE years INT;
    ->     SET years = TIMESTAMPDIFF(YEAR, join_date, CURDATE());
    ->
    ->     IF years < 2 THEN
    ->         RETURN 'Fresher';
    ->     ELSEIF years BETWEEN 2 AND 5 THEN
    ->         RETURN 'Junior';
    ->     ELSEIF years BETWEEN 6 AND 10 THEN
    ->         RETURN 'Mid';
    ->     ELSE
    ->         RETURN 'Senior';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT emp_id, experience_category(join_date) AS category
    -> FROM emp_salary;
+--------+----------+
| emp_id | category |
+--------+----------+
|      1 | Junior   |
|      2 | Junior   |
|      3 | Mid      |
|      4 | Senior   |
+--------+----------+
4 rows in set (0.00 sec)
```

5. Create a late fee calculator using function where late fee of ₹50/day is applicable after due date limited to as maximum amount of ₹1000.

```
DELIMITER //

CREATE FUNCTION calculate_late_fee(due DATE, returned DATE)
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE days INT;
  DECLARE fee INT;

  IF returned <= due THEN
```

```
    RETURN 0;
  END IF;

  SET days = DATEDIFF(returned, due);
  SET fee = days * 50;

  IF fee > 1000 THEN
    SET fee = 1000;
  END IF;

  RETURN fee;
END //

DELIMITER ;


SELECT record_id,
    calculate_late_fee(due_date, return_date) AS late_fee
FROM fee_details;
```

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION calculate_late_fee(due DATE, returned DATE)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE days INT;
    ->     DECLARE fee INT;
    ->
    ->     IF returned <= due THEN
    ->         RETURN 0;
    ->     END IF;
    ->
    ->     SET days = DATEDIFF(returned, due);
    ->     SET fee = days * 50;
    ->
    ->     IF fee > 1000 THEN
    ->         SET fee = 1000;
    ->     END IF;
    ->
    ->     RETURN fee;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT record_id,
    ->        calculate_late_fee(due_date, return_date) AS late_fee
    -> FROM fee_details;
+-----------+----------+
| record_id | late_fee |
+-----------+----------+
|         1 |      200 |
|         2 |     1000 |
+-----------+----------+
2 rows in set (0.00 sec)
```